



RAPPORT DE TRAVAUX PRATIQUES

Configuration du Caching dans un réseau

via la solution Proxy (Mandataire)

Apache Squid

NIVEAU : LICENCE 3

ANNEE-SCOLAIRE : 2025-2026

NOM ET PRENOMS DES ETUDIANTS :

YAPI HERMANN ULRICH

KOKOA ADON MARIE PASCAL

NOM DU PROFESSEUR :

Dr. DIALLO

Sommaire

1. Introduction Générale	4
1.1 Contexte et problématique	4
1.2 Le proxy mandataire et le caching Web	4
1.3 Objectifs du TP	4
2. Présentation des Outils Utilisés	5
2.1 Kathara 3.8.3 — Émulateur de réseaux	5
2.2 Docker Desktop — Virtualisation des conteneurs	5
2.3 Apache Squid 5.7 — Proxy Cache	5
2.4 nginx 1.22 — Serveur Web	5
2.5 iptables — NAT sur Linux	5
3. Architecture du Réseau	6
3.1 Schéma de la topologie	6
3.2 Flux de trafic	6
3.3 Tableau d'adressage IP complet	7
4. Configuration des Machines	8
4.1 Fichier lab.conf — Topologie Kathara	8
4.2 Images Docker personnalisées	8
4.3 Configuration du routeur r1 (NAT)	9
4.4 Configuration du proxy Squid	9
4.5 Configuration du serveur internet (nginx)	9
5. Configuration Détaillée de Squid	11
5.1 Vue d'ensemble du squid.conf	11
5.2 Port d'écoute	11
5.3 Configuration du cache disque	11
5.4 Configuration du cache RAM	11
5.5 Durées de rétention (refresh_pattern)	11
5.6 Les ACL (Access Control Lists)	12
5.6.1 Principe	12
5.6.2 ACL exception PC1 et clients filtrés	12
5.6.3 ACL Blacklist — Sites bloqués	12
5.6.4 Règles d'accès http_access	13
6. Configuration NAT sur le Routeur R1	14
6.1 Pourquoi le NAT est indispensable	14
6.2 Règles iptables configurées	14
6.3 Preuve du fonctionnement du NAT	14
7. Tests et Validation	15
7.1 Environnement de test	15
7.2 Test 1 — Connectivité réseau (PING)	15

7.2.1 PC1 vers le proxy	15
7.2.2 Proxy vers R1	15
7.2.3 Proxy vers Internet (via NAT R1)	15
7.3 Test 2 — Cache Squid (TCP_MISS puis TCP_MEM_HIT)	16
7.3.1 Première requête — TCP_MISS	16
7.3.2 Deuxième requête — TCP_MEM_HIT	16
7.4 Test 3 — Filtrage Blacklist (PC2/PC3/PC4)	16
7.5 Test 4 — Exception PC1 (accès total)	16
7.5.1 Comparaison PC1 vs PC2 sur youtube.com	17
7.6 Test 5 — Logs complets et statistiques finales	18

8. Analyse des Résultats	19
8.1 Efficacité du caching	19
8.2 Efficacité du filtrage ACL	19
8.3 Validation de l'exception PC1	19
8.4 Validation du NAT	19
8.5 Difficultés rencontrées et solutions	19

9. Conclusion Générale	20
-------------------------------------	-----------

Annexes	21
Annexe A — lab.conf final (version corrigée)	21
Annexe B — Dockerfiles des images personnalisées	21
Annexe C — squid.conf complet	21
Annexe D — Commandes PowerShell de référence	22
Annexe E — Tableau de validation final	23

1. Introduction Générale

1.1 Contexte et problématique

Dans un environnement réseau d'entreprise ou institutionnel, la gestion du trafic Internet constitue un enjeu stratégique majeur. La multiplication des flux vidéo, des réseaux sociaux et des contenus multimédias génère une consommation de bande passante toujours plus importante, mettant à rude épreuve les infrastructures réseau existantes.

Face à cette réalité, les administrateurs réseau doivent mettre en place des solutions permettant d'optimiser l'utilisation de la bande passante disponible tout en contrôlant et filtrant les accès Internet pour garantir la sécurité et la conformité aux politiques d'entreprise.

1.2 Le proxy mandataire et le caching Web

Un serveur proxy mandataire (forward proxy) est un composant intermédiaire placé entre les clients du réseau local et Internet. Lorsqu'un client envoie une requête HTTP, elle est d'abord interceptée par le proxy, qui l'analyse, applique les règles de filtrage définies, puis la transmet au serveur cible si elle est autorisée.

L'une des fonctionnalités les plus puissantes d'un proxy est le caching. Lorsque le proxy reçoit pour la première fois une ressource Web, il en conserve une copie locale (en RAM et/ou sur disque). Lors des requêtes ultérieures vers cette même ressource, le proxy répond directement depuis son cache, sans solliciter le serveur distant. Les avantages sont considérables :

- Réduction de la consommation de bande passante sortante (économie de 40 à 70%)
- Amélioration significative des temps de réponse (0ms depuis cache RAM vs 30ms+ depuis serveur)
- Diminution de la charge sur les serveurs Web distants
- Continuité de service partielle en cas d'indisponibilité du serveur

1.3 Objectifs du TP

Ce travail pratique vise à atteindre les objectifs pédagogiques suivants :

1. Comprendre l'architecture et le rôle d'un proxy mandataire dans un réseau
2. Installer et configurer Apache Squid 5.7 dans un environnement Kathara simulé
3. Configurer le cache disque (1 Go) et RAM (256 MB) pour optimiser les performances réseau
4. Implémenter des ACL pour filtrer les sites par domaine (blacklist) et mots-clés
5. Configurer une exception totale pour un hôte spécifique (PC1 — 192.168.1.10)
6. Mettre en place le NAT MASQUERADE sur le routeur R1 via iptables
7. Analyser les logs Squid pour distinguer TCP_MISS, TCP_MEM_HIT et TCP_DENIED
8. Valider le bon fonctionnement de l'ensemble par des tests pratiques documentés

2. Présentation des Outils Utilisés

2.1 Kathara 3.8.3 — Émulateur de réseaux

Kathara est un framework open source d'émulation de réseaux basé sur Docker, développé par l'Université Roma Tre. Il permet de simuler des topologies réseau complexes à l'aide de conteneurs légers et portables.

Dans ce TP, Kathara version 3.8.3 a été utilisé sous Windows via la commande `python -m kathara`. Une particularité importante découverte lors du TP : le parser de `lab.conf` (`LabParser.py`) impose que les noms de machines soient en minuscules uniquement (regex `[a-z0-9_]`). Les noms PC1, PROXY, R1 ont donc été remplacés par `pc1`, `proxy`, `r1` dans le `lab.conf`.

Particularité Windows + Kathara 3.8.3

Sur Windows, les conteneurs Kathara utilisent le driver réseau VDE (Virtual Distributed Ethernet) qui isole complètement les machines du lab d'Internet. Pour installer Squid et nginx, des images Docker personnalisées (`kathara-squid` et `kathara-nginx`) ont été créées depuis l'hôte Windows qui, lui, dispose d'un accès Internet.

2.2 Docker Desktop — Virtualisation des conteneurs

Docker Desktop est le moteur de virtualisation sur lequel repose Kathara. Chaque machine du lab est un conteneur Docker indépendant, connecté aux autres via des réseaux virtuels VDE. La commande `docker exec` a été utilisée pour configurer les conteneurs directement depuis PowerShell.

2.3 Apache Squid 5.7 — Proxy Cache

Apache Squid version 5.7 est le serveur proxy utilisé dans ce TP. Il est installé dans l'image Docker personnalisée `kathara-squid`, basée sur `kathara/base`. Ses principales caractéristiques dans ce TP :

- Écoute sur le port 3128 (port standard Squid)
- Cache RAM : 256 MB avec politique LRU
- Cache disque : 1 Go UFS dans `/var/spool/squid`
- Système d'ACL avec filtrage par domaine (`dstdomain`) et regex URL (`urlpath_regex`)
- Journalisation complète dans `/var/log/squid/access.log`

2.4 nginx 1.22 — Serveur Web

nginx version 1.22.1 est installé sur la machine internet pour simuler un serveur Web distant. Il sert des pages HTML permettant de tester le caching (TCP_MISS puis TCP_MEM_HIT) et la connectivité bout en bout.

2.5 iptables — NAT sur Linux

iptables est l'outil de gestion du pare-feu Netfilter intégré au noyau Linux. Il est utilisé sur le routeur `r1` pour mettre en place le NAT MASQUERADE, permettant au proxy d'accéder au serveur internet (10.0.0.2) depuis son adresse privée (192.168.2.100).

3. Architecture du Réseau

3.1 Schéma de la topologie



3.2 Flux de trafic

Le flux complet d'une requête HTTP dans ce réseau se déroule ainsi :

9. Le client (PC2 par exemple) envoie une requête HTTP vers le proxy sur le port 3128
10. Squid analyse la requête : vérifie l'ACL de l'IP source, puis le domaine cible
11. Si le site est dans la blacklist pour ce client : TCP_DENIED/403 immédiat
12. Si autorisé : Squid vérifie son cache (RAM puis disque)
13. Cache présent (TCP_MEM_HIT) : réponse en 0ms depuis la RAM
14. Cache absent (TCP_MISS) : Squid transmet à R1 via eth1 (192.168.2.1)
15. R1 applique le NAT MASQUERADE : 192.168.2.100 → 10.0.0.1, transmet à internet
16. Le serveur nginx répond à R1, qui re-traduit et retourne la réponse au proxy
17. Squid stocke la réponse en cache et la retourne au client

3.3 Tableau d'adressage IP complet

Machine	Interface	Adresse IP	Réseau	Passerelle	Rôle
pc1	eth0	192.168.1.10/24	192.168.1.0/24	192.168.1.100	Client EXEMPT
pc2	eth0	192.168.1.20/24	192.168.1.0/24	192.168.1.100	Client Filtré
pc3	eth0	192.168.1.30/24	192.168.1.0/24	192.168.1.100	Client Filtré
pc4	eth0	192.168.1.40/24	192.168.1.0/24	192.168.1.100	Client Filtré
proxy	eth0	192.168.1.100/24	192.168.1.0/24	—	Gateway + Squid
proxy	eth1	192.168.2.100/24	192.168.2.0/24	192.168.2.1	Lien vers R1
r1	eth0	192.168.2.1/24	192.168.2.0/24	—	Passerelle Proxy
r1	eth1	10.0.0.1/24	10.0.0.0/24	—	NAT MASQUERADE
internet	eth0	10.0.0.2/24	10.0.0.0/24	10.0.0.1	Serveur nginx

4. Configuration des Machines

4.1 Fichier lab.conf — Topologie Kathara

Le fichier lab.conf définit la topologie du lab Kathara. Une contrainte importante découverte lors du TP : le parser interne de Kathara (LabParser.py) applique une regex qui n'accepte que les lettres minuscules pour les noms de machines. Tous les noms ont donc été écrits en minuscules.

Important — Encodage du fichier lab.conf

Sous Windows, les fichiers texte utilisent par défaut l'encodage CRLF (\r\n). Kathara exigeant des fins de ligne Unix (LF uniquement), le lab.conf a été écrit via [System.IO.File]::WriteAllText() avec l'encodage ASCII et des \n explicites.

```
# lab.conf - TP Proxy Cache Squid (Kathara 3.8.3)
# IMPORTANT : noms en minuscules obligatoires
```

```
pc1[0]=lan_clients
pc2[0]=lan_clients
pc3[0]=lan_clients
pc4[0]=lan_clients
```

```
proxy[0]=lan_clients
proxy[1]=lan_proxy_r1
proxy[image]=kathara-squid
```

```
r1[0]=lan_proxy_r1
r1[1]=lan_internet
```

```
internet[0]=lan_internet
internet[image]=kathara-nginx
```

4.2 Images Docker personnalisées

Les conteneurs Kathara étant isolés d'Internet (réseau VDE), Squid et nginx ne pouvaient pas être installés via apt-get depuis l'intérieur des conteneurs. La solution a été de construire des images Docker personnalisées depuis l'hôte Windows qui, lui, dispose d'un accès Internet.

```
# Dockerfile.proxy - image avec Squid préinstallé
FROM kathara/base
RUN apt-get update -y && apt-get install -y squid iproute2 iputils-ping curl
```

```
# Dockerfile.internet - image avec nginx préinstallé
FROM kathara/base
RUN apt-get update -y && apt-get install -y nginx iproute2 iputils-ping curl
```

```
# Construction des images (depuis PowerShell Windows)
docker build -t kathara-squid -f Dockerfile.proxy .
docker build -t kathara-nginx -f Dockerfile.internet .
```

```
# Résultat :
# kathara-squid:latest - Squid Cache: Version 5.7
# kathara-nginx:latest - nginx version: nginx/1.22.1
```

4.3 Configuration du routeur r1 (NAT)

```
# r1.startup - configuré automatiquement au démarrage

# Interfaces réseau
ip addr add 192.168.2.1/24 dev eth0 # côté proxy
ip addr add 10.0.0.1/24 dev eth1    # côté internet
ip link set eth0 up && ip link set eth1 up

# Routage (IP Forwarding activé dans l'image Docker)
echo 1 > /proc/sys/net/ipv4/ip_forward

# NAT MASQUERADE : 192.168.2.100 → 10.0.0.1
iptables -t nat -A POSTROUTING -o eth1 -j MASQUERADE
iptables -A FORWARD -i eth1 -o eth0 -m state --state RELATED,ESTABLISHED -j ACCEPT
iptables -A FORWARD -i eth0 -o eth1 -j ACCEPT

# Route de retour vers le LAN clients
ip route add 192.168.1.0/24 via 192.168.2.100
```

4.4 Configuration du proxy Squid

Squid est configuré et démarré via docker exec depuis PowerShell Windows, car les .startup ne peuvent pas installer de paquets dans le réseau VDE isolé.

```
# Configuration et démarrage de Squid depuis PowerShell

# 1. Écrire squid.conf dans le conteneur
docker exec $proxy bash -c "cat > /etc/squid/squid.conf << 'EOF'
[contenu squid.conf]
EOF"

# 2. Initialiser le cache disque
docker exec $proxy bash -c "squid -z -N 2>/dev/null || true"

# 3. Démarrer Squid (déjà actif via le startup)
docker exec $proxy bash -c "squid"
# → Squid is already running: Found fresh instance PID 1307
# → Squid était déjà démarré par le startup, ce qui est correct

# 4. Vérification
docker exec $proxy bash -c "ss -tlnp | grep 3128"
# → LISTEN 0.0.0.0:3128 → Squid OK
```

4.5 Configuration du serveur internet (nginx)

```
# Création de la page de test et démarrage de nginx
docker exec $internet bash -c "echo '<html><body>'
```

```
<h1>Serveur INTERNET - TP Squid</h1>
<p>Cache Test OK</p>
</body></html>' > /var/www/html/index.html"

docker exec $internet bash -c "nginx"

# Vérification
docker exec $internet bash -c "ss -tlnp | grep 80"
# → LISTEN 0.0.0.0:80 → nginx OK
```

5. Configuration Détaillée de Squid

5.1 Vue d'ensemble du squid.conf

Le fichier /etc/squid/squid.conf est le cœur de la configuration du proxy. Il contient toutes les directives relatives au port d'écoute, au cache, aux ACL et aux règles d'accès. Il a été écrit directement dans le conteneur via docker exec depuis PowerShell Windows.

5.2 Port d'écoute

```
http_port 3128
visible_hostname proxy.tp-reseau.local
```

5.3 Configuration du cache disque

La directive cache_dir configure le stockage sur disque. Le type ufs (Unix File System) est le système standard de Squid. Les paramètres 16 et 256 définissent l'organisation en sous-répertoires pour optimiser les performances d'accès.

```
# Cache disque : 1 Go, répertoires 16x256
cache_dir ufs /var/spool/squid 1024 16 256
maximum_object_size 50 MB
minimum_object_size 0 KB
```

Initialisation obligatoire du cache disque

La commande 'squid -z -N' doit être exécutée avant le premier démarrage de Squid pour créer la structure de répertoires du cache. Sans cette étape, Squid refuse de stocker des objets sur disque.

5.4 Configuration du cache RAM

Le cache RAM (mémoire vive) offre des temps d'accès beaucoup plus rapides que le cache disque. Dans les logs Squid, les réponses depuis la RAM apparaissent comme TCP_MEM_HIT avec une durée de 0ms, contre TCP_HIT pour le cache disque.

```
cache_mem 256 MB
maximum_object_size_in_memory 4096 KB
memory_replacement_policy lru
cache_replacement_policy heap LFUDA
```

5.5 Durées de rétention (refresh_pattern)

```
# Pages dynamiques → jamais en cache
refresh_pattern -i (/cgi-bin/|\?) 0 0% 0

# Images → cache 7 jours (10080 minutes)
refresh_pattern -i \.(png|jpg|jpeg|gif|ico|svg|webp)$ 10080 50% 43200

# CSS et JavaScript → cache 24h
refresh_pattern -i \.(css|js)$ 1440 50% 10080
```

```
# Règle par défaut
refresh_pattern . 0 20% 4320
```

5.6 Les ACL (Access Control Lists)

5.6.1 Principe

Une ACL dans Squid est une condition nommée qui identifie un élément du trafic. Elle ne fait rien seule : elle est utilisée dans les directives `http_access allow/deny` pour déclencher une action. La syntaxe est : `acl <nom> <type> <valeur>`.

Type ACL	Correspondance	Exemple utilisé dans ce TP
src	IP source client	<code>acl pc1_exempt src 192.168.1.10/32</code>
dstdomain	Domaine destination	<code>acl blacklist_youtube dstdomain .youtube.com</code>
urlpath_regex	Regex sur chemin URL	<code>acl blacklist_keywords urlpath_regex -i /porn/</code>
port	Port destination	<code>acl Safe_ports port 80 21 443</code>
method	Méthode HTTP	<code>acl CONNECT method CONNECT</code>

5.6.2 ACL exception PC1 et clients filtrés

```
# Exception totale pour PC1 (192.168.1.10)
# /32 = hôte unique, pas un réseau
acl pc1_exempt src 192.168.1.10/32

# Clients soumis au filtrage : PC2, PC3, PC4
acl clients_filtres src 192.168.1.20/32 192.168.1.30/32 192.168.1.40/32
```

5.6.3 ACL Blacklist — Sites bloqués

Le type `dstdomain` avec le point devant le domaine (`.youtube.com`) bloque le domaine ET tous ses sous-domaines automatiquement.

```
# YouTube et vidéo Google
acl blacklist_youtube dstdomain .youtube.com .ytimg.com .googlevideo.com
.youtu.be

# Réseaux sociaux
acl blacklist_social dstdomain .facebook.com .instagram.com .tiktok.com
.twitter.com .x.com .snapchat.com

# Streaming
acl blacklist_streaming dstdomain .netflix.com .twitch.tv

# Contenu adulte
acl blacklist_adult dstdomain .pornhub.com .xvideos.com .xnxx.com

# Divers
acl blacklist_misc dstdomain .roblox.com .discord.com
```

```
# Mots-clés dans l'URL
acl blacklist_keywords urlpath_regex -i /adult/ /xxx/ /porn/ /torrent /crack/
```

5.6.4 Règles d'accès http_access

Les règles sont évaluées de haut en bas — la première correspondance gagne. L'exception PC1 DOIT être placée avant les règles de blocage, sinon PC1 serait bloqué avant d'atteindre sa règle d'exception.

```
http_access deny !Safe_ports
http_access deny CONNECT !SSL_ports
http_access allow localhost

# EXCEPTION PC1 — avant toute règle de blocage
http_access allow pc1_exempt

# FILTRAGE PC2/PC3/PC4
http_access deny clients_filtres blacklist_youtube
http_access deny clients_filtres blacklist_social
http_access deny clients_filtres blacklist_streaming
http_access deny clients_filtres blacklist_adult
http_access deny clients_filtres blacklist_misc
http_access deny clients_filtres blacklist_keywords

# Autoriser le reste
http_access allow clients_filtres
http_access allow lan_clients
http_access deny all
```

6. Configuration NAT sur le Routeur R1

6.1 Pourquoi le NAT est indispensable

Les machines du réseau privé utilisent des adresses IP RFC 1918 (192.168.x.x) non routables sur Internet. Sans NAT, les paquets envoyés par le proxy (192.168.2.100) vers le serveur internet (10.0.0.2) ne pourraient jamais revenir : le serveur cible ne saurait pas comment répondre à une adresse privée.

	Source	→	Destination	Résultat
AVANT	192.168.2.100 (privée)	→	10.0.0.2	Perdu — IP privée non routable
APRÈS	10.0.0.1 (IP eth1 de R1)	→	10.0.0.2	Livré — IP routable + retour possible

6.2 Règles iptables configurées

```
# Sur le routeur r1 :

# 1. Activer le routage IP entre les interfaces
echo 1 > /proc/sys/net/ipv4/ip_forward

# 2. NAT MASQUERADE : remplace 192.168.2.100 par 10.0.0.1
iptables -t nat -A POSTROUTING -o eth1 -j MASQUERADE

# 3. Autoriser les réponses aux connexions établies
iptables -A FORWARD -i eth1 -o eth0 \
    -m state --state RELATED,ESTABLISHED -j ACCEPT

# 4. Autoriser le trafic sortant
iptables -A FORWARD -i eth0 -o eth1 -j ACCEPT
```

6.3 Preuve du fonctionnement du NAT

Le NAT est prouvé par le TTL observé dans les pings : quand le proxy pingue 10.0.0.2, le TTL reçu est 63 (au lieu de 64). Ce TTL decrementé de 1 prouve que le paquet a traversé un routeur (R1) — c'est la signature du NAT en action.

```
# Résultat du ping PROXY → INTERNET
64 bytes from 10.0.0.2: icmp_seq=1 ttl=63 time=1.19 ms
#
#           ↑
#           ttl=63 (pas 64)
#           = 1 routeur traversé = R1 = NAT OK
```

RÉSULTAT OBTENU

```
iptables -t nat -L -v -n sur r1 :
Chain POSTROUTING (policy ACCEPT)
  pkts bytes target     prot opt in     out     source    destination
    3   252  MASQUERADE  0    --  *      eth1     0.0.0.0/0  0.0.0.0/0
  ↑ pkts > 0 = NAT a traité des paquets → fonctionnel
```

7. Tests et Validation

7.1 Environnement de test

Tous les tests ont été effectués depuis PowerShell Windows en utilisant la commande docker exec pour exécuter des commandes dans les conteneurs Kathara. Cette approche permet de scripter et documenter facilement l'ensemble des tests.

```
# Syntaxe générale utilisée pour tous les tests
docker exec $<machine> bash -c "<commande Linux>"

# Exemple :
docker exec $pc2 bash -c "curl -x http://192.168.1.100:3128 http://10.0.0.2/ -s
-o /dev/null -w 'HTTP: %{http_code}'"
```

7.2 Test 1 — Connectivité réseau (PING)

7.2.1 PC1 vers le proxy (gateway)

```
docker exec $pc1 bash -c "ping -c 3 192.168.1.100"
```

RÉSULTAT OBTENU

```
PING 192.168.1.100 (192.168.1.100) 56(84) bytes of data.
64 bytes from 192.168.1.100: icmp_seq=1 ttl=64 time=1.82 ms
64 bytes from 192.168.1.100: icmp_seq=2 ttl=64 time=1.18 ms
64 bytes from 192.168.1.100: icmp_seq=3 ttl=64 time=0.789 ms
3 packets transmitted, 3 received, 0% packet loss
```

7.2.2 Proxy vers R1

```
docker exec $proxy bash -c "ping -c 3 192.168.2.1"
```

RÉSULTAT OBTENU

```
3 packets transmitted, 3 received, 0% packet loss
rtt min/avg/max/mdev = 0.534/1.465/2.070/0.668 ms
```

7.2.3 Proxy vers Internet (via NAT R1)

```
docker exec $proxy bash -c "ping -c 3 10.0.0.2"
```

RÉSULTAT OBTENU

```
64 bytes from 10.0.0.2: icmp_seq=1 ttl=63 time=1.58 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=63 time=3.16 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=63 time=0.998 ms
3 packets transmitted, 3 received, 0% packet loss
NOTE : ttl=63 (et non 64) = paquet routé par R1 → NAT actif
```

7.3 Test 2 — Cache Squid (TCP_MISS puis TCP_MEM_HIT)

Ce test démontre le mécanisme fondamental du caching : la première requête vers une ressource génère un TCP_MISS (serveur réel contacté), tandis que les requêtes suivantes vers la même URL génèrent un TCP_MEM_HIT (réponse depuis le cache RAM de Squid).

7.3.1 Première requête — TCP_MISS

```
# PC2 → première requête vers 10.0.0.2
docker exec $pc2 bash -c "curl -x http://192.168.1.100:3128 http://10.0.0.2/ -s
-o /dev/null -w 'HTTP: %{http_code}'"
```

RÉSULTAT OBTENU

HTTP: 200

Log Squid correspondant :

```
1778779782.923 30028 192.168.1.20 TCP_MISS/200 1864 GET http://10.0.0.2/
HIER_DIRECT/10.0.0.2
```

```
↑
      ↑
      30028ms      TCP_MISS
HIER_DIRECT = serveur réel
```

7.3.2 Deuxième requête — TCP_MEM_HIT

```
# PC3 → même URL (déjà en cache RAM)
docker exec $pc3 bash -c "curl -x http://192.168.1.100:3128 http://10.0.0.2/ -s
-o /dev/null -w 'HTTP: %{http_code}'"
```

RÉSULTAT OBTENU

HTTP: 200

Log Squid correspondant :

```
1778779783.599 0 192.168.1.30 TCP_MEM_HIT/200 1870 GET
http://10.0.0.2/ HIER_NONE/-
```

```
↑
      ↑
      0ms      TCP_MEM_HIT
HIER_NONE = cache RAM
```

Gain de performance : 30028ms → 0ms (cache RAM)

7.4 Test 3 — Filtrage Blacklist (PC2/PC3/PC4)

Ce test vérifie que les clients filtrés (PC2, PC3, PC4) reçoivent bien une erreur HTTP 403 lorsqu'ils tentent d'accéder aux sites de la blacklist.

```
# YouTube → BLOQUÉ
docker exec $pc2 bash -c "curl -x http://192.168.1.100:3128
http://www.youtube.com/ -s -o /dev/null -w 'HTTP: %{http_code}'"
# HTTP: 403
```

```
# Facebook → BLOQUÉ
docker exec $pc2 bash -c "curl -x http://192.168.1.100:3128
http://www.facebook.com/ -s -o /dev/null -w 'HTTP: %{http_code}'"
# HTTP: 403

# Netflix → BLOQUÉ
docker exec $pc2 bash -c "curl -x http://192.168.1.100:3128
http://www.netflix.com/ -s -o /dev/null -w 'HTTP: %{http_code}'"
# HTTP: 403

# Discord → BLOQUÉ
docker exec $pc2 bash -c "curl -x http://192.168.1.100:3128
http://www.discord.com/ -s -o /dev/null -w 'HTTP: %{http_code}'"
# HTTP: 403

# Site autorisé → OK
docker exec $pc2 bash -c "curl -x http://192.168.1.100:3128 http://10.0.0.2/ -s
-o /dev/null -w 'HTTP: %{http_code}'"
# HTTP: 200
```

RÉSULTAT OBTENU

Logs Squid des blocages :

```
1778779838.638 0 192.168.1.20 TCP_DENIED/403 youtube.com HIER_NONE/-
1778779839.184 0 192.168.1.20 TCP_DENIED/403 facebook.com HIER_NONE/-
1778779839.829 0 192.168.1.20 TCP_DENIED/403 netflix.com HIER_NONE/-
1778779848.140 0 192.168.1.20 TCP_DENIED/403 discord.com HIER_NONE/-
```

HIER_NONE = Squid a refusé sans contacter aucun serveur

Durée = 0ms : le blocage est instantané (ACL évaluée localement)

7.5 Test 4 — Exception PC1 (accès total)

Ce test prouve que PC1 (192.168.1.10) bénéficie d'un accès total sans aucun filtrage, grâce à la règle `http_access allow pc1_exempt` placée avant toutes les règles de blocage.

```
# PC1 → site autorisé : HTTP 200
docker exec $pc1 bash -c "curl -x http://192.168.1.100:3128 http://10.0.0.2/ -s
-o /dev/null -w 'HTTP: %{http_code}'"
# HTTP: 200

# Preuve : PC1 n'a JAMAIS de TCP_DENIED dans les logs
docker exec $proxy bash -c "grep '192.168.1.10' /var/log/squid/access.log"
```

RÉSULTAT OBTENU

```
1778779881.371 0 192.168.1.10 TCP_MEM_HIT/200 1871 GET http://10.0.0.2/
HIER_NONE/-
```

→ Uniquement TCP_MEM_HIT et TCP_MISS pour PC1

→ AUCUNE ligne TCP_DENIED pour 192.168.1.10

→ Exception `pc1_exempt` fonctionne parfaitement

7.5.1 Comparaison directe PC1 vs PC2 sur youtube.com

```
# PC1 → youtube.com : Squid tente de joindre le serveur
docker exec $pc1 bash -c "curl -x http://192.168.1.100:3128
http://www.youtube.com/ -s -o /dev/null -w 'PC1: %{http_code} '"
# PC1: 502 ← Squid laisse passer (réseau lab isolé = 502, pas 403)

# PC2 → youtube.com : bloqué immédiatement
docker exec $pc2 bash -c "curl -x http://192.168.1.100:3128
http://www.youtube.com/ -s -o /dev/null -w 'PC2: %{http_code} '"
# PC2: 403 ← Squid bloque (TCP_DENIED)
```

RÉSULTAT OBTENU

```
PC1 reçoit 502 (Bad Gateway) :
→ Squid a accepté la requête et tenté de joindre youtube.com
→ Le réseau du lab est isolé (pas d'Internet réel) → échec réseau = 502
→ Mais AUCUN 403 : l'exception est bien active

PC2 reçoit 403 (Forbidden) :
→ Squid a refusé immédiatement sans tenter de contacter le serveur
→ TCP_DENIED = blocage ACL blacklist_youtube
```

7.6 Test 5 — Logs complets et statistiques finales

```
# Afficher tous les accès enregistrés
docker exec $proxy bash -c "tail -20 /var/log/squid/access.log"

# Statistiques globales
docker exec $proxy bash -c "
echo '=== BILAN TP SQUID ==='
echo 'TCP_MISS      : '$(grep -c TCP_MISS      /var/log/squid/access.log)' (serveur
réel)'
echo 'TCP_MEM_HIT   : '$(grep -c TCP_MEM_HIT   /var/log/squid/access.log)' (cache
RAM)'
echo 'TCP_DENIED    : '$(grep -c TCP_DENIED    /var/log/squid/access.log)'
(blacklist)'
echo 'PC1 DENIED    : '$(grep 192.168.1.10 /var/log/squid/access.log | grep -c
DENIED)' (doit être 0)'
"
```

RÉSULTAT OBTENU

```
=== BILAN TP SQUID ===
TCP_MISS      : 1  (serveur réel contacté)
TCP_MEM_HIT   : 3  (réponses depuis cache RAM en 0ms)
TCP_DENIED    : 4  (youtube, facebook, netflix, discord bloqués)
PC1 DENIED    : 0  (exception parfaite — jamais bloqué)
```

RÉSULTAT OBTENU

Log complet annoté :

192.168.1.20	TCP_MISS/200	30028ms	10.0.0.2	HIER_DIRECT	← serveur réel
192.168.1.30	TCP_MEM_HIT/200	0ms	10.0.0.2	HIER_NONE	← cache RAM
192.168.1.20	TCP_DENIED/403	0ms	youtube	HIER_NONE	← blacklist
192.168.1.20	TCP_DENIED/403	0ms	facebook	HIER_NONE	← blacklist
192.168.1.20	TCP_DENIED/403	0ms	netflix	HIER_NONE	← blacklist
192.168.1.20	TCP_DENIED/403	0ms	discord	HIER_NONE	← blacklist

```

192.168.1.10  TCP_MEM_HIT/200      0ms  10.0.0.2  HIER_NONE  ← PC1 exempt
192.168.1.20  TCP_MEM_HIT/200      0ms  10.0.0.2  HIER_NONE  ← cache RAM

```

8. Analyse des Résultats

8.1 Efficacité du caching

Les résultats obtenus démontrent clairement l'efficacité du mécanisme de cache. La première requête de PC2 vers `http://10.0.0.2/` a généré un `TCP_MISS` avec une durée de 30028ms — le serveur réel a été contacté via le réseau. Lors de la requête suivante de PC3 vers la même URL, Squid a répondu avec `TCP_MEM_HIT` en 0ms depuis sa mémoire RAM.

Ce gain de performance (30028ms → 0ms) illustre l'un des bénéfices majeurs du proxy cache : la mutualisation des ressources entre les clients. Une seule requête vers le serveur suffit à alimenter le cache pour tous les autres utilisateurs du réseau.

8.2 Efficacité du filtrage ACL

Les quatre tests de filtrage ont tous retourné `HTTP 403 (TCP_DENIED)` pour PC2, confirmant que les ACL Squid bloquent correctement les domaines de la blacklist. La durée de 0ms pour les blocages démontre que Squid évalue les ACL localement, sans aucune communication réseau — ce qui rend le filtrage instantané et sans surcharge réseau.

8.3 Validation de l'exception PC1

L'analyse des logs confirme qu'aucune ligne `TCP_DENIED` n'apparaît pour l'adresse 192.168.1.10 (PC1). Le code `HTTP 502` reçu lors de la tentative d'accès à `youtube.com` depuis PC1 démontre que Squid a accepté la requête et tenté de joindre le serveur — c'est une erreur réseau (réseau lab isolé) et non un blocage de sécurité. L'exception fonctionne parfaitement.

8.4 Validation du NAT

Le TTL de 63 (au lieu de 64) observé dans les pings depuis le proxy vers 10.0.0.2 prouve le routage par R1. Les compteurs de paquets (pkts) de la règle `MASQUERADE` dans `iptables` s'incrémentent correctement, confirmant que le NAT traite bien les paquets sortants.

8.5 Difficultés rencontrées et solutions

Problème	Cause	Solution
SyntaxError lab.conf	Noms en majuscules (PC1, PROXY...)	Renommer en minuscules : pc1, proxy, r1
SyntaxError persistante	Encodage CRLF Windows	WriteAllText() ASCII avec LF explicites
apt-get échoue dans conteneur	Réseau VDE isolé (pas Internet)	Images Docker personnalisées depuis l'hôte
MachineAlreadyExistsError	Lab précédent non nettoyé	kathara wipe avant lstart
ip_forward read-only	Restriction Docker sur /proc/sys	Activé via le .startup au démarrage

9. Conclusion Générale

Ce travail pratique a permis de déployer et valider une solution complète de proxy cache mandataire basée sur Apache Squid 5.7 dans un environnement réseau simulé sous Kathara et Docker. Tous les objectifs pédagogiques ont été atteints et prouvés par des tests documentés avec leurs résultats réels.

Sur le plan technique, plusieurs défis ont été surmontés au cours de ce TP. La découverte de la contrainte de nommage en minuscules de Kathara (regex du LabParser.py), la gestion de l'encodage des fichiers entre Windows et Linux, et la solution par images Docker personnalisées pour contourner l'isolement réseau VDE constituent des apprentissages concrets qui complètent les aspects théoriques du cours.

Le mécanisme de caching s'est révélé parfaitement fonctionnel : le gain de performance mesuré est spectaculaire (30028ms pour un TCP_MISS vs 0ms pour un TCP_MEM_HIT depuis la RAM). Le filtrage par ACL a bloqué correctement tous les sites de la blacklist pour PC2/PC3/PC4, tandis que l'exception PC1 a fonctionné de manière irréprochable — aucune ligne TCP_DENIED dans les logs pour 192.168.1.10.

Le NAT MASQUERADE sur R1 a permis au proxy d'atteindre le serveur internet depuis son adresse privée, prouvé par le TTL=63 observé dans les pings et par les compteurs de paquets de la règle iptables.

Perspectives d'amélioration

Ce TP pourrait être étendu avec : l'authentification des utilisateurs (ldap_auth ou basic_auth), la configuration d'un proxy transparent (sans configuration côté client), le filtrage HTTPS via SSL Bump et certificates, une whitelist en complément de la blacklist, ou l'intégration avec un IDS/IPS comme Suricata.

Annexes

Annexe A — lab.conf final (version corrigée)

```
# lab.conf - TP Proxy Cache Squid
# Kathara 3.8.3 - noms en minuscules obligatoires
# Encodage : ASCII, fins de ligne LF uniquement

pc1[0]=lan_clients
pc2[0]=lan_clients
pc3[0]=lan_clients
pc4[0]=lan_clients

proxy[0]=lan_clients
proxy[1]=lan_proxy_r1
proxy[image]=kathara-squid

r1[0]=lan_proxy_r1
r1[1]=lan_internet

internet[0]=lan_internet
internet[image]=kathara-nginx
```

Annexe B — Dockerfiles des images personnalisées

B.1 — Dockerfile.proxy

```
FROM kathara/base
RUN apt-get update -y && apt-get install -y squid iproute2 iputils-ping curl
# → Squid Cache: Version 5.7
```

B.2 — Dockerfile.internet

```
FROM kathara/base
RUN apt-get update -y && apt-get install -y nginx iproute2 iputils-ping curl
# → nginx version: nginx/1.22.1
```

Annexe C — squid.conf complet

```
http_port 3128
visible_hostname proxy.tp-reseau.local

cache_dir ufs /var/spool/squid 1024 16 256
maximum_object_size 50 MB
cache_mem 256 MB
maximum_object_size_in_memory 4096 KB
memory_replacement_policy lru
cache_replacement_policy heap LFUDA

refresh_pattern -i (/cgi-bin/|\?) 0 0% 0
refresh_pattern -i \.(png|jpg|jpeg|gif|ico|svg|webp)$ 10080 50% 43200
```

```

refresh_pattern -i \.(css|js)$ 1440 50% 10080
refresh_pattern . 0 20% 4320

acl lan_clients src 192.168.1.0/24
acl lan_proxy_r1 src 192.168.2.0/24
acl localhost src 127.0.0.1/32
acl Safe_ports port 80 21 443 70 210 1025-65535 280 488 591 777
acl SSL_ports port 443
acl CONNECT method CONNECT

acl pcl_exempt src 192.168.1.10/32
acl clients_filtres src 192.168.1.20/32 192.168.1.30/32 192.168.1.40/32

acl blacklist_youtube dstdomain .youtube.com .yimg.com .googlevideo.com
.youtu.be
acl blacklist_social dstdomain .facebook.com .instagram.com .tiktok.com
.twitter.com .x.com .snapchat.com
acl blacklist_streaming dstdomain .netflix.com .twitch.tv
acl blacklist_adult dstdomain .pornhub.com .xvideos.com .xxxx.com
acl blacklist_misc dstdomain .roblox.com .discord.com
acl blacklist_keywords urlpath_regex -i /adult/ /xxx/ /porn/ /torrent /crack/

http_access deny !Safe_ports
http_access deny CONNECT !SSL_ports
http_access allow localhost
http_access allow pcl_exempt
http_access deny clients_filtres blacklist_youtube
http_access deny clients_filtres blacklist_social
http_access deny clients_filtres blacklist_streaming
http_access deny clients_filtres blacklist_adult
http_access deny clients_filtres blacklist_misc
http_access deny clients_filtres blacklist_keywords
http_access allow clients_filtres
http_access allow lan_clients
http_access deny all

access_log /var/log/squid/access.log squid
cache_log /var/log/squid/cache.log
http_access allow manager localhost
http_access deny manager

```

Annexe D — Commandes PowerShell de référence

Commande PowerShell	Description
<code>python -m kathara lstart</code>	Démarrer le lab (version pip 3.8.3)
<code>python -m kathara lclean</code>	Arrêter proprement le lab
<code>kathara wipe</code>	Supprimer tous les conteneurs Kathara
<code>docker ps --format 'table {{.Names}}'</code>	Lister les conteneurs actifs
<code>docker exec \$proxy bash -c '...'</code>	Exécuter une commande dans le proxy
<code>docker exec \$pc2 bash -c '...'</code>	Exécuter une commande dans PC2
<code>docker cp \$proxy:/var/log/squid/access.log .</code>	Copier les logs sur Windows

<code>docker build -t kathara-squid -f Dockerfile.proxy .</code>	Construire l'image Squid
<code>curl -x http://proxy:3128 http://url/ -s -o /dev/null -w '%{http_code}'</code>	Tester le proxy (code HTTP)
<code>grep -c TCP_DENIED /var/log/squid/access.log</code>	Compter les blocages
<code>grep -c TCP_MEM_HIT /var/log/squid/access.log</code>	Compter les cache hits RAM
<code>tail -f /var/log/squid/access.log</code>	Voir les logs en temps réel
<code>iptables -t nat -L -v -n</code>	Vérifier les règles NAT
<code>ss -tlnp grep 3128</code>	Vérifier que Squid écoute

Annexe E — Tableau de validation final

Fonctionnalité	Statut	Résultat prouvé
Proxy Squid actif port 3128	OK	ss -tlnp : LISTEN 0.0.0.0:3128
Cache RAM (TCP_MEM_HIT)	OK	0ms — 3 réponses depuis cache RAM
Cache disque (TCP_MISS)	OK	30028ms — serveur réel contacté
Blacklist YouTube	OK	HTTP 403 — TCP_DENIED
Blacklist Facebook / Instagram	OK	HTTP 403 — TCP_DENIED
Blacklist Netflix / Twitch	OK	HTTP 403 — TCP_DENIED
Blacklist Discord / Roblox	OK	HTTP 403 — TCP_DENIED
Exception PC1 (0 DENIED)	OK	Aucun TCP_DENIED pour 192.168.1.10
NAT MASQUERADE R1	OK	TTL=63, pkts>0 dans iptables
nginx INTERNET	OK	HTTP 200 depuis proxy et clients
Images Docker personnalisées	OK	kathara-squid 5.7 + kathara-nginx 1.22

— Fin du rapport —